

REST API / АЛГОРИТМ для регистрации пользователя

МАРШАЛОВ АНДРЕЙ

Оглавление

REST API для регистрации пользователя	3
1. HTTP Метод и URL	3
2. Входные параметры	3
3. Выходные параметры при успешном ответе.....	3
4. Выходные параметры при ответе с ошибкой	3
5. Описание ошибок.....	3
6. Пример запроса	4
7. Пример ответа	5
Алгоритм регистрации пользователя	6
Пошаговый процесс:	6
Диаграмма алгоритма:	7

REST API для регистрации пользователя

1. HTTP Метод и URL

- **Метод:** POST
- **URL:** /api/register

2. Входные параметры

Параметр	Тип данных	Обязательность	Ограничения
firstName	string	required	Минимальная длина: 1 символ, максимальная длина: 50 символов.
lastName	string	required	Минимальная длина: 1 символ, максимальная длина: 50 символов.
username	string	required	Уникальное имя пользователя, длина от 3 до 50 символов.
password	string	required	Минимум 8 символов

3. Выходные параметры при успешном ответе

Параметр	Тип данных	Обязательность	Описание
userId	string	required	Уникальный идентификатор пользователя.
username	string	required	Имя пользователя.
message	string	required	Сообщение об успешной регистрации.

4. Выходные параметры при ответе с ошибкой

Параметр	Тип данных	Обязательность	Описание
errorCode	string	required	Код ошибки.
message	string	required	Сообщение об ошибке.

5. Описание ошибок

Код ошибки	Тип ошибки	Сообщение
400	Bad Request	Ошибка валидации данных. Например, "Password must have at least one non-alphanumeric character."
409	Conflict	Пользователь с таким именем уже существует.
500	Internal Server Error	Внутренняя ошибка сервера.

6. Пример запроса

openapi: 3.0.0

info:

title: Book Store Registration API

description: API для регистрации пользователей в Book Store

version: 1.0.0

servers:

- url: https://api.bookstore.example.com/v1

description: Основной сервер

paths:

/api/register:

post:

summary: Регистрация нового пользователя

description: Создаёт нового пользователя в системе Book Store

requestBody:

required: true

content:

application/json:

schema:

\$ref: '#/components/schemas/UserRegistrationRequest'

responses:

'201':

description: Пользователь успешно зарегистрирован

content:

application/json:

schema:

\$ref: '#/components/schemas/UserRegistrationResponse'

'400':

description: Ошибка валидации данных

content:

application/json:

schema:

\$ref: '#/components/schemas/ErrorResponse'

'409':

description: Пользователь с таким именем уже существует

content:

application/json:

schema:

\$ref: '#/components/schemas/ErrorResponse'

'500':

description: Внутренняя ошибка сервера

content:

application/json:

schema:

\$ref: '#/components/schemas/ErrorResponse'

```
components:
  schemas:
    UserRegistrationRequest:
      type: object
      required:
        - firstName
        - lastName
        - username
        - password
      properties:
        firstName:
          type: string
          description: Имя пользователя
          minLength: 1
          maxLength: 50
          example: "Ivan"
        lastName:
          type: string
          description: Фамилия пользователя
          minLength: 1
          maxLength: 50
          example: "Ivanov"
        username:
          type: string
          description: Уникальное имя пользователя
          minLength: 3
          maxLength: 50
          example: "ivan"
        password:
          type: string
          description: Пароль пользователя
          minLength: 8
          example: "Password1!"
```

7. Пример ответа

Успешный сценарий:

```
UserRegistrationResponse:
  type: object
  properties:
    userId:
      type: string
      description: Уникальный идентификатор пользователя
```

example: "123e4567-e89b-12d3-a456-426614174000"
username:
 type: string
 description: Имя пользователя
 example: "ivan"
message:
 type: string
 description: Сообщение об успешной регистрации
 example: "User registered successfully"

Сценарий с ошибкой:

ErrorResponse:
 type: object
 properties:
 errorCode:
 type: string
 description: Код ошибки
 example: "VALIDATION_ERROR"
 message:
 type: string
 description: Сообщение об ошибке
 example: "Password must have at least one non-alphanumeric character, one digit, one uppercase, one lowercase, and be at least 8 characters long."

Алгоритм регистрации пользователя

Пошаговый процесс:

- Получение запроса на регистрацию:**
 - Принять POST-запрос на `/api/register` с телом запроса в формате JSON.
 - Проверить наличие всех обязательных полей: `firstName`, `lastName`, `username`, `password`.
- Валидация данных:**
 - Проверить, что `firstName` и `lastName` содержат только буквы и имеют длину от 1 до 50 символов.
 - Проверить, что `username` уникален, содержит только буквы и цифры, и его длина от 3 до 50 символов.
 - Проверить, что `password` соответствует требованиям:
 - Минимум 8 символов.
- Проверка уникальности username:**
 - Запросить базу данных на наличие пользователя с таким же `username`.
 - Если пользователь найден, вернуть ошибку с кодом 409 и сообщением: "User already exists".
- Хеширование пароля:**

- Захешировать пароль с использованием безопасного алгоритма (например, bcrypt).
- 5. **Сохранение пользователя в базе данных:**
 - Сохранить нового пользователя в базе данных с хешированным паролем.
- 6. **Формирование ответа:**
 - Если все проверки пройдены успешно, вернуть ответ с кодом 201 и данными пользователя:
 - Если произошла ошибка валидации, вернуть ответ с кодом 400 и сообщением об ошибке.
 - Если произошла внутренняя ошибка сервера, вернуть ответ с кодом 500 и сообщением об ошибке.

Диаграмма алгоритма:

